## How to run the baseline (step by step)

**What you are doing and why.** The original paper (de Padua and Shang, MSR 2018) built a model that predicts which Java files get post-release bugs. RQ2 asks: does adding "exception-handling anti-patterns" to the usual metrics make the model better? The authors shared their R scripts and data. We run them ourselves and check that we get the same answer. If we do, the baseline is "reproduced".

You do not need to know R. You copy commands into a window and read a few numbers at the end.

Time needed: about 1 hour the first time (installing is the slow part). The model run itself took under 10 minutes on our Mac.

---

### Step 1. Install R and RStudio

- R is the programming language. RStudio is the window you type into. Install R first, then RStudio.
- R: go to cran.r-project.org, click "Download R for macOS" (or Windows), and install. On a Mac with an M1/M2/M3/M4 chip, choose the **arm64** installer.
- RStudio Desktop: posit.co/download/rstudio-desktop. Open it from Applications.
- Check: in RStudio's Console (bottom left), type `R.version.string` and press Enter. It should print `R version 4.x.x`.
- Why RStudio and not the plain R app: RStudio includes a tool called Pandoc that the notebooks need to make their reports. The plain R app does not, and fails with "pandoc version 2.8 or higher is required".

### Step 2. Download the authors' package

- Go to https://guipadua.github.io/eh-model-defects2018/ and download "Input data, scripts, and output". Unzip it. You get a folder `eh-model-defects2018_scripts_with_input_data` containing two notebooks (`0-DataSourcesAnalysis.Rmd`, `1-Model_Construction.Rmd`), two helper scripts, an `input` folder (the data) and an `output` folder (the authors' saved results, files ending `_BKP.csv`).

### Step 3. Replace one helper script (fixes a known error)

- The authors' script fails on current R (R 4.6) at line 270 of `analysis_functions.R` with a long error that ends in a "condition has length > 1" type message. We fixed it with one change: `!is.na(refit)` became `!identical(refit, NA)`.

- Easiest: copy `analysis_functions.R` from our repo's `baseline` folder over the one in the package folder.
- Or edit line 270 yourself.
- Do not change anything else. The professor wants the analysis settings untouched.

### Step 4. Point R at the folder and install the packages

Paste each block into RStudio's **Console** and press Enter. (If R asks to choose a CRAN mirror in a pop-up and shows an error, the first line below fixes that.)

Set the folder (change the path to where you unzipped it):

```r
setwd("/Users/YOURNAME/Downloads/eh-model-defects2018_scripts_with_input_data")
list.files()
```

You should see the two `.Rmd` files, the two `.R` files, `input` and `output`.

Install the packages (the first time takes 10 to 20 minutes; if it asks "install from sources?" answer `no`):

```r
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages(c("rms","plotly","Hmisc","e1071","caret","logistf","rmarkdown"))
install.packages("BiodiversityR")
```

BiodiversityR is the one most likely to cause trouble. The notebooks only load it with `require()`, which gives a warning instead of stopping, and nothing in the code uses it. If it will not install, skip it and write that in your log.

### Step 5. Run notebook 0 (prepare the data)

```r
rmarkdown::render("0-DataSourcesAnalysis.Rmd")
```

What happens: it reads the five CSV files in `input/`, joins them per project (Hadoop, Hibernate, Umbraco), drops files with missing data, and saves `0-all_no_missing.RData`. That file is not in the package, so **notebook 0 must run first**. Success looks like `Output created: 0-DataSourcesAnalysis.html`.

### Step 6. Run notebook 1 (build and test the models)

```r
rmarkdown::render("1-Model_Construction.Rmd")
```

What happens: for each project it builds the models, simplifies them, and tests them with a bootstrap (it re-runs the model 1000 times on random re-samples). The console may look frozen. That is normal. Keep RStudio open and the computer awake. It finishes when `>` comes back and you see `Output created: 1-Model_Construction.html`. It writes `output/base_test_1.csv` and `output/base_test_2.csv`.

### Step 7. Read the results and compare

Open `output/base_test_2.csv` (files with catch blocks only; this is RQ2). Open `output/base_test_2_BKP.csv` (the authors' saved run) beside it.

Look at these rows (column `name`) and `project`:

- `CAT.BASE` = the model with the usual metrics (size, complexity, change history, authors, comments).
- `CAT.BSAP` = the same model **plus** the anti-pattern metrics.
- Projects to look at: `hadoop-2.6` and `hibernate-5.0` (ignore Umbraco, which is C#).

Columns to compare:

- **`r2_c_r`**: how well the final, simplified model explains defects (0 to 1, higher is better), corrected so it is not too optimistic. This matches the paper's Table 4 ("optimism-reduced Nagelkerke R-squared on the simplified model").
  - r2 = score; c = corrected for over-optimism by the bootstrap; r = after simplifying (refit).
- **`signifcant_r`** (spelled that way in the file): the metrics that stayed in the final model, so are statistically significant.
- **`coef`**: the size and sign of each effect. Positive = more of this goes with more defects after release.

What we got (yours may differ in the third decimal because the bootstrap has no fixed random seed):

| Project | Model | Paper | Authors' file | Our run |
|---|---|---|---|---|
| Hadoop | BASE | 0.33 | 0.329 | 0.329 |
| Hadoop | BSAP | 0.35 | 0.352 | 0.348 |
| Hibernate | BASE | 0.21 | 0.213 | 0.213 |
| Hibernate | BSAP | 0.23 | 0.235 | 0.232 |

Significant anti-patterns in BSAP (all with positive coefficients): Hadoop: Ignoring Interrupted Exception, Log and Throw, Potentially Recoverable. Hibernate: Catch SLOC, Dummy Handler.

**It counts as reproduced** if the same anti-patterns are significant in the same direction and R-squared is close. That is what the professor asked for. Exact numbers can differ a little.

### Step 8. Keep a change log

Write down every problem you hit and what you did, even small ones. Example entries from our run:

1. CRAN mirror pop-up failed in the plain R app: set the mirror in code with `options(repos=...)`.
2. Plain R app has no Pandoc: ran the notebooks from RStudio.
3. BiodiversityR did not load: left as is (not used by the code).
4. Notebook 1 failed at `analysis_functions.R` line 270 under R 4.6: changed `!is.na(refit)` to `!identical(refit, NA)`.

Save your environment too:

```r
writeLines(capture.output(sessionInfo()), "sessionInfo.txt")
```

### Send us

`output/base_test_2.csv`, `sessionInfo.txt`, and your change log. Put them in your own branch of the repo (for example `baseline-yourname`) and open a pull request.

### If something fails

Copy the **exact** error text (the last 10 lines are usually enough) into the group chat, and say which step you were on.